FOLLOW UP 1 Deep Neural Networks (DNNs)

1. **Cree un Notebook en NotebookLM de Google, agregue la presentación hecha en clase como fuente de conocimiento y encuentre respuesta a las siguientes preguntas:**

- ¿Qué garantiza y qué no garantiza el teorema de aproximación universal?

> El **Teorema de Aproximación Universal** (demostrado por George Cybenko en 1989 y Kurt Hornik en 1991) establece las bases teóricas sobre la capacidad de representación de las redes neuronales alimentadas hacia adelante (*feedforward*) Lo que SÍ garantiza el teorema:
>
> - **Existencia de una solución:** Garantiza que una red neuronal con una capa oculta y suficientes neuronas (o un número arbitrario de capas) tiene la capacidad teórica de **aproximar cualquier función continua** con el nivel de precisión deseado.
> - **Capacidad de representación:** Asegura que las redes neuronales son lo suficientemente flexibles para modelar, ajustar o separar una amplia variedad de distribuciones de datos en tareas de regresión o clasificación cuando se les asignan los pesos adecuados.
>
> Lo que NO garantiza el teorema:
>
> - **No enseña cómo calcular los pesos:** Es un **resultado de existencia**demuestra que la configuración de pesos existe, pero no proporciona un procedimiento o algoritmo para encontrarlos en la práctica.
> - **No determina la arquitectura requerida:** No especifica cuántas neuronas o capas ocultas exactas se necesitan para lograr la aproximación deseada.
> - **No garantiza la generalización:** Que una red tenga la capacidad teórica de ajustar una función no significa que vaya a generalizar bien con datos nuevos o fuera de su rango de entrenamiento.
> - **No contempla restricciones del mundo real:** El teorema asume condiciones teóricas e ignora limitaciones prácticas como tiempo de entrenamiento acotado, capacidad computacional limitada, datos insuficientes o errores en los conjuntos de datos.

- ¿Por qué no basta con minimizar la pérdida de entrenamiento al elegir el tamaño de la red?

> No basta con minimizar la pérdida (*loss*) de entrenamiento para seleccionar el tamaño de una red neuronal porque **Una mayor capacidad de ajuste no implica una mayor capacidad de generalización**.
>
> Si únicamente nos guiamos por la pérdida de entrenamiento para elegir la anchura y profundidad de la red, nos enfrentamos a los siguientes problemas clave:
>
> 1. **Sobreajuste (Overfitting) y memorización del ruido**:
>
>> - A medida que se incrementa el número de neuronas o capas, la red gana capacidad y puede representar funciones cada vez más complejas, lo que reduce progresivamente la pérdida de entrenamiento
>> - Sin embargo, cuando la red es demasiado grande para la cantidad de datos disponibles, deja de aprender la tendencia o patrón general y comienza a **memorizar el ruido**

específico de la muestra de entrenamiento.

2. Ampliación de la brecha de generalización

- **Comportamiento divergente:** Mientras que la pérdida de entrenamiento continúa bajando hacia valores muy pequeños al aumentar el tamaño de la red, el error en datos nuevos no vistos (prueba o validación) tiende a formar una **curva en forma de U**.
- Al sobrepasar la capacidad adecuada, el error de prueba vuelve a subir, provocando un aumento drástico en la **brecha de generalización** (la diferencia entre el error de prueba y el de entrenamiento)

3. Existencia del error irreducible

- Los datos del mundo real suelen contener ruido inherente que ningún modelo puede o debe predecir, conocido como **error irreducible**
- Intentar forzar la pérdida de entrenamiento a bajar más allá de esta cota inferior solo conduce a ajustar las fluctuaciones aleatorias del conjunto de datos

¿Cómo se debe seleccionar el tamaño de la red?

La arquitectura y el tamaño de la red se deben elegir evaluando el desempeño en un conjunto de **validación** independizado del de entrenamiento, buscando el punto donde el error de validación alcance su mínimo

- ¿Cuándo preferiría F1 o AP antes que la exactitud? Dé un ejemplo de su propio trabajo

La **exactitud (accuracy)** se utiliza principalmente cuando las clases del problema están **balanceadas** y todos los tipos de error tienen un costo equivalente. Sin embargo, en escenarios donde existe un desbalance de clases o costos asimétricos entre errores, la exactitud resulta engañosa. **¿Cuándo preferir el F1-Score?**

- **Clases desbalanceadas:** Se prefiere el **F1-Score** (la media armónica entre precisión y exhaustividad) cuando la clase de interés es minoritaria y se necesita un balance directo entre evitar falsos positivos y minimizar falsos negativos para un umbral determinado.
- **Evitar la ilusión de la exactitud:** En conjuntos desbalanceados, un modelo ingenuo que prediga siempre la clase mayoritaria alcanzará una exactitud muy alta pero será inútil en la práctica; el F1-Score penaliza este comportamiento asignándole un valor de cero.

¿Cuándo preferir la Precisión Promedio (AP)?

- **Evaluación sin fijar umbrales:** La **Precisión Promedio (Average Precision o AP)**, calculada a partir de la curva de precisión-exhaustividad, se prefiere cuando se quiere evaluar el rendimiento global del modelo ordenando las probabilidades predichas, sin atarse a un umbral de decisión fijo.
- **Baja prevalencia de positivos:** AP es la métrica más informativa cuando la proporción de casos positivos es sumamente baja, ya que mide el comportamiento del modelo respecto a la prevalencia base del problema.

Ejemplo del material del curso En las diapositivas de la materia se presenta un experimento sobre un conjunto de prueba con $n = 600$ muestras, donde solo 23 son positivos reales (prevalencia del 3.8%):

- Clasificador trivial ("siempre negativo"): Obtiene una exactitud del 96.2% (0.962), pero una exhaustividad y un F1 de 0.
- Red neuronal con umbral 0.5: Logra una exactitud del 96.5%, pero con una exhaustividad de solo 0.26 y un F1 de 0.36.
- Ajuste de umbral a 0.39 (elegido en validación): La exhaustividad sube a 0.48 y el F1 mejora a 0.43, aunque la exactitud baje ligeramente a 0.952.
- Evaluación con AP: La AP alcanza un valor de 0.46 frente a la línea de base de prevalencia de 0.04, reflejando con precisión la capacidad del modelo para priorizar los verdaderos positivos independientemente del umbral.

Este enfoque es fundamental en tareas del curso como la detección y segmentación geoespacial (por ejemplo, identificación de cultivos o elementos específicos en imágenes satelitales), donde las clases de interés ocupan un porcentaje reducido del área total.

- ¿Cómo cambia el tamaño óptimo de la red si se duplica el conjunto de datos?

Al duplicar el conjunto de datos de entrenamiento (n), el tamaño óptimo de la red cambia en función de la capacidad actual del modelo y del comportamiento de la brecha de generalización:

1. **Permite aumentar el tamaño óptimo (mayor capacidad)**

- **Reducción de la brecha de generalización:** Al contar con un mayor número de muestras, la brecha entre el error de prueba y el de entrenamiento se reduce considerablemente para modelos de mayor anchura o profundidad.
- **Mayor aprovechamiento de parámetros:** Arquitecturas con más parámetros (w más grande), que antes caían en sobreajuste o presentaban alta varianza debido a la escasez de datos, ahora pueden entrenarse de forma estable y acercarse al piso de error irreducible (σ)

2. **Comportamiento según la condición inicial de la red**

- **Si la red sufría subajuste (demasiado pequeña):** Si la red era muy pequeña para la complejidad del problema, se estanca en un error alto por un sesgo elevado. En este caso, más datos no corrigen el problema por sí solos; se requiere incrementar obligatoriamente el tamaño o la capacidad de la red.
- **Si la red ya estaba en el límite de error irreducible:** Si el modelo previo ya contaba con la capacidad suficiente para la función y su curva de error de validación ya se había aplanado cerca del límite de ruido inherente, duplicar los datos aportará un beneficio marginal y el tamaño óptimo de la red se mantendrá constante.

3. **Criterio de diagnóstico empírico**

- No existe una regla fija universal (como la heurística tradicional de "10 datos por parámetro", la cual no siempre se cumple en la práctica).
- La decisión de escalar el tamaño de la red tras aumentar el dataset se debe evaluar mediante las curvas de aprendizaje en validación: si la brecha de generalización se vuelve muy estrecha y la curva sigue bajando, es señal de que se puede incrementar la anchura o profundidad de la red para extraer más información.

- ¿Por qué no basta una sola semilla para comparar dos arquitecturas?

No basta con utilizar una sola **semilla aleatoria (random seed)** para comparar dos arquitecturas de redes neuronales porque el entrenamiento de una red involucra múltiples procesos estocásticos, lo que provoca que los resultados con una sola semilla sean ruidosos y puedan llevar a conclusiones engañosas

Las razones principales incluyen:

1. La semilla controla elementos estocásticos clave

Fijar la semilla establece el estado del generador pseudoaleatorio que determina:

- **La inicialización de los pesos:** La superficie de pérdida (*loss landscape*) no es convexa y tiene múltiples mínimos locales. Distintas semillas ubican a la red en puntos de partida diferentes, lo que puede llevar al optimizador a soluciones o mínimos de calidad muy distinta.
- **El orden de los mini-lotes (mini-batches):** Cambiar la semilla altera la secuencia en la que se presentan los datos al optimizador (como SGD o Adam), modificando la trayectoria del gradiente estocástico.
- **La partición de los datos:** Si la división en entrenamiento, validación y prueba es aleatoria, la semilla condiciona qué ejemplos específicos caen en cada subconjunto.

2. Riesgo de confundir "suerte" con superioridad arquitectónica Una arquitectura podría parecer superior a otra simplemente por haber tenido una inicialización excepcionalmente favorable en esa semilla específica.

- **Ejemplo del material de clase:** En los experimentos con la red 1-3-1, solo 1 de cada 20 semillas logró alcanzar un RMSE inferior a 0.2, mientras que la mediana fue de 0.45. Evaluar el modelo únicamente con esa semilla "afortunada" habría dado una falsa impresión de su verdadero desempeño.

3. Las arquitecturas tienen distinta variabilidad o sensibilidad

- La dispersión de los resultados entre semillas no es uniforme entre todas las arquitecturas. Por ejemplo, en redes con más capas (más profundas), la dispersión entre semillas suele ser considerablemente mayor debido a la complejidad del proceso de optimización.
- Una sola semilla oculta la **estabilidad** de una arquitectura: una red puede tener un mejor caso promedio pero una alta varianza, mientras que otra puede ser sumamente consistente a través de diferentes inicializaciones.

4. Buena práctica: medir tendencia central y dispersión

- Evaluar mediante **múltiples semillas** (por ejemplo, 10 o 20 semillas) permite observar la forma clara de las curvas de error (como la curva en U al variar el tamaño o la anchura) que con una sola semilla resultaría demasiado ruidosa.
- La práctica estándar consiste en reportar la **media (o mediana)** acompañada de su **dispersión** (desviación estándar o rango), evaluando el comportamiento típico de la arquitectura y no el mejor caso puntual.

- ¿Qué pasa si la distribución estadística de los datos tiene dos picos? ¿Qué alternativa propondría?

> Cuando la distribución estadística de los datos presenta dos picos (una distribución bimodal o multimodal), el impacto depende de si el fenómeno se presenta en el espacio de características/cobertura de datos o en la variable de salida (y). **1. Si ocurre en la cobertura de las características o datos de entrenamiento**
>
> Si los datos se concentran en dos regiones o picos separados por un hueco o valle intermedio sin muestras:
>
> > - **¿Qué pasa?** La red neuronal puede obtener un **error muy bajo en los datos usados**, pero tendrá un **error sumamente alto en las zonas intermedias sin datos**. El diagnóstico es una **cobertura insuficiente de la distribución**.
> > - **Alternativas propuestas:**
> >
> > > - **Muestreo en los huecos:** Recolectar o generar datos en las regiones intermedias para cubrir los vacíos entre los picos.
> >
> > > - **Partición estratificada / por zonas:** Diseñar divisiones de validación y prueba por bloques o estratos para asegurarse de evaluar la generalización del modelo tanto en los picos como fuera de ellos.
>
> **2. Si la variable de salida (y) es bimodal para una misma entrada (x)** Si ante una misma entrada existen dos respuestas válidas posibles (dos picos de probabilidad):
>
> > - **¿Qué pasa?** Si se entrena un modelo de regresión estándar con **Error Cuadrático Medio (ECM/MSE)**, la red intentará predecir la esperanza matemática (el promedio condicional). En una distribución bimodal, el promedio cae en el valle central entre los dos picos, produciendo una predicción que no corresponde a ninguna de las dos soluciones reales y resulta inutilizable.
> > - **Alternativas propuestas:**
> >
> > > - **Redes de Densidad Mixta (Mixture Density Networks - MDN):** Modificar la capa de salida para que la red prediga los parámetros de una mezcla de distribuciones (p. ej., medias, varianzas y pesos de dos o más gaussianas) en lugar de un único valor escalar.
> > > - **Formulación como Clasificación:** Discretizar el rango continuo de salida en intervalos (bins) y utilizar entropía cruzada con Softmax4, lo que permite a la red asignar probabilidades altas a ambos picos de manera independiente.
> > > - **Modelos Generativos (p. ej., GANs):** Implementar redes generativas compuestas capaces de aprender y muestrear distribuciones multimodales complejas5.

- ¿Cuántos datos necesita su problema? ¿Cómo lo estimaría con una curva de aprendizaje?

> No existe una cantidad universal ni una regla fija predeterminada de cuántos datos necesita un problema. El volumen suficiente depende de tres factores principales: la complejidad de la función objetivo, el nivel de ruido en los datos (error irreducible) y la capacidad o tamaño de la red neuronal. Aunque históricamente se ha mencionado la regla heurística de "10 datos por parámetro", en la práctica esta no es una ley rígida y redes complejas pueden alcanzar un buen rendimiento con

proporciones mucho menores de datos por parámetro.Para estimar empíricamente la cantidad de datos requerida para su problema específico, el diagnóstico estándar consiste en construir una curva de aprendizaje evaluando la métrica de error en función del número de datos de entrenamiento (n). Cómo estimar la cantidad de datos con la curva de aprendizaje

1. Construcción experimental de la curva:

- Entrene la arquitectura seleccionada utilizando subconjuntos de datos progresivamente mayores (por ejemplo, $n = 10, 20, 50, 100, 500, \ldots$).
- Grafique en el eje horizontal el tamaño del conjunto de entrenamiento (n) y en el eje vertical el error (o métrica) tanto para el conjunto de entrenamiento como para el de validación/prueba.

- Para evitar guiarse por fluctuaciones aleatorias de una sola corrida, evalúe y promedie el resultado sobre múltiples semillas aleatorias.

2. Interpretación del diagnóstico de la curva:

- Caso 1: La curva de validación se aplana (alcanza una asíntota)

  - Diagnóstico: El error de validación deja de disminuir, la brecha de generalización (diferencia entre prueba y entrenamiento) se vuelve muy estrecha y la curva se estabiliza cerca del error irreducible σ (el ruido inherente a los datos).

  - Estimación: Los datos actuales son suficientes2. Agregar más ejemplos tendrá un impacto marginal insignificante en el desempeño.

- Caso 2: La curva de validación sigue decreciendo y la brecha es amplia

  - Diagnóstico: Si la brecha entre el error de prueba y de entrenamiento es grande, pero la curva de validación mantiene una pendiente descendente constante a medida que aumenta n, el modelo aún padece de varianza y se beneficiará de más datos.

  - Estimación: Se puede extrapolar la trayectoria descendente de la curva de validación hacia un valor de error objetivo para proyectar cuántos datos adicionales se necesitan recolectar.

- Caso 3: Ambas curvas se estancan en un error elevado (subajuste)

  - Diagnóstico: Si tanto el error de entrenamiento como el de validación se aplanan en un nivel de error alto, la red es demasiado pequeña o simple para representar la complejidad de la función.
  - Estimación: Aumentar el número de datos no resolverá el problema. Primero se debe incrementar la capacidad de la red (su anchura o profundidad) antes de reevaluar la necesidad de más datos.

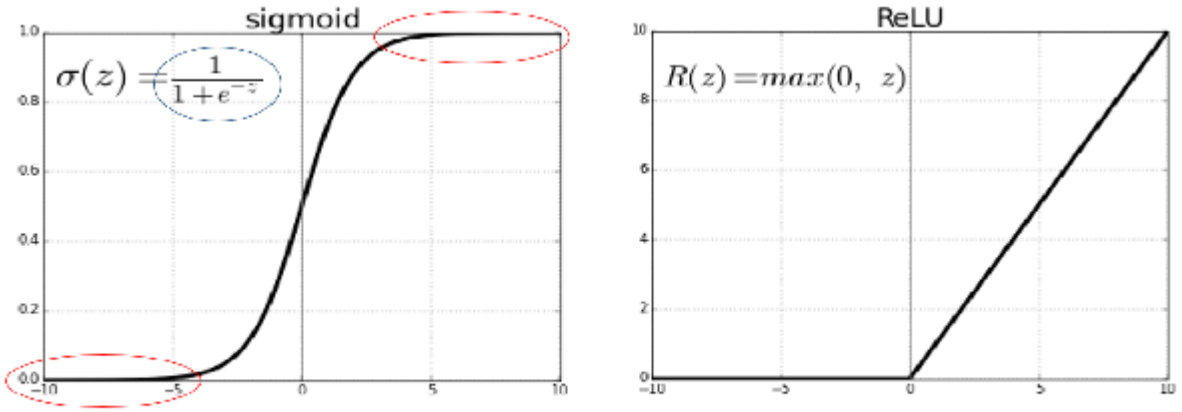
- La permutación dice que una entrada «importa»: ¿qué puede concluir y qué no puede concluir?

> La importancia por permutación es un método de explicabilidad global que evalúa cuánto aumenta el error del modelo cuando se barajan aleatoriamente los valores de una columna de entrada para romper su relación con la variable objetivo y.
>
> Lo que SÍ se puede concluir:
>
> > - Grado de uso por parte del modelo: Determina cuánto depende el modelo entrenado de esa variable específica para realizar sus predicciones.
> > - Detección de características prescindibles: Permite identificar entradas inútiles o de ruido puro, indicando qué variables se pueden eliminar de la red sin impactar negativamente el desempeño.
> > - Sensibilidad global del modelo: Proporciona una métrica agregada sobre cómo responde la arquitectura ajustada ante la pérdida de información de cada característica en el conjunto de datos.
>
> Lo que NO se puede concluir:
>
> > - No prueba causalidad en el mundo real: La métrica describe exclusivamente el funcionamiento del modelo ajustado, no una relación de causa y efecto en el fenómeno real.
> > - No refleja importancia real si se mide en entrenamiento: Si se calcula sobre el conjunto de entrenamiento en un modelo sobreajustado, una variable de ruido puede parecer falsamente importante; por ello, debe medirse obligatoriamente en los conjuntos de validación o prueba.
> > - No aísla la importancia en presencia de correlación: Cuando existen variables de entrada fuertemente correlacionadas entre sí, la importancia tiende a repartirse o diluirse entre ellas, pues barajar una variable no destruye la información que la red aún puede extraer de su variable redundante23.No es un resultado único ni definitivo: Dos redes neuronales con un nivel de rendimiento equivalente (o entrenadas con semillas aleatorias diferentes) pueden generar atribuciones de importancia por permutación distintas para las mismas entradas.

2. Activation Functions

Lea los artículos propuestos adelante, busque más información si es necesario y debata con sus compañeros:

2.1 Por qué la función de transferencia o de activación famosa llamada ReLu es mejor que la Sigmoide?



a. Zonas de gradiente cero
b. Función más costosa computacionalmente

1. Previene el desvanecimiento del gradiente (Vanishing Gradient) Como se discute en el foro de Data Science, la derivada de la función Sigmoide alcanza un máximo de tan solo 0.25. Durante la fase de retropropagación (backpropagation) en redes de múltiples capas, los gradientes se multiplican entre sí. Al multiplicar repetidamente fracciones menores a 1, el gradiente se encoge drásticamente hasta volverse casi cero. Cuando esto ocurre, las capas iniciales de la red neuronal dejan de actualizarse y el aprendizaje se estanca.

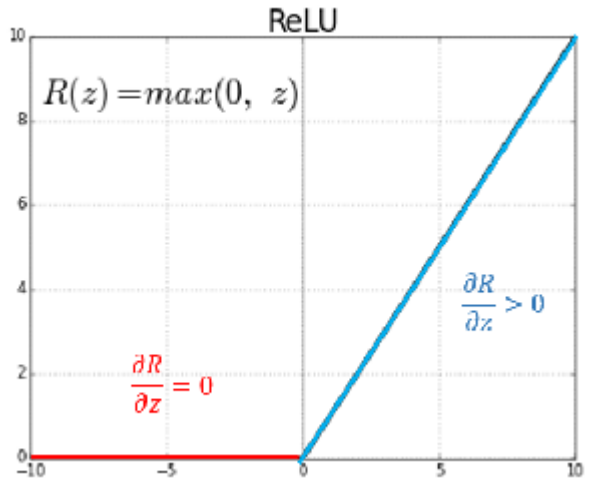
ReLU soluciona esto porque su derivada es exactamente 1 para todos los valores positivos, permitiendo que la señal de error fluya inalterada a lo largo de redes muy profundas.

2. Aceleración masiva del entrenamiento El artículo de AlexNet de la Universidad de Toronto introdujo la idea de que ReLU es una función "no saturada". El estudio documentó empíricamente que las redes neuronales profundas con unidades ReLU se entrenan de manera significativamente más ágil. En sus pruebas de laboratorio con el dataset CIFAR-10, una red neuronal alcanzó un objetivo del 25% de error de entrenamiento 6 veces más rápido utilizando ReLU en comparación con la misma arquitectura usando una función saturada tradicional como Tanh o Sigmoide.

3. Eficiencia computacional La matemática detrás de la Sigmoide involucra un cálculo exponencial intensivo: f(x) = 1 / (1 + e^-x). En contraste, ReLU es una operación aritmética extremadamente sencilla que solo requiere comparar un número con cero: f(x) = max(0, x). Al eliminar las operaciones exponenciales, el modelo requiere mucho menos tiempo de CPU/GPU para ejecutarse, lo cual es vital cuando se tiene una arquitectura de millones de parámetros.

4. Induce dispersión en el modelo (Sparsity) ReLU establece un valor absoluto de 0 para cualquier entrada negativa. Esto introduce un concepto de dispersión; significa que en un momento dado, un gran porcentaje de las neuronas en la red están literalmente "apagadas" y no emiten señal. Este comportamiento crea modelos más ligeros y obliga a las neuronas que sí están activas a procesar únicamente características fuertemente relevantes. Por el contrario, la función Sigmoide casi siempre arroja un valor decimal positivo, manteniendo a prácticamente toda la red ligeramente activa al mismo tiempo.

2.2 Por qué ReLu es no lineal si es rectilínea, o acaso si es?



> La prueba matemática Para que una función sea considerada verdaderamente "lineal" en el álgebra, debe cumplir con el principio de superposición: f(a + b) debe dar exactamente el mismo resultado que f(a) + f(b).
>
> Veamos qué pasa con ReLU, cuya operación es f(x) = max(0, x):
>
> > Si tomamos a = -1 y b = 1, evaluamos por separado:
> >
> > - f(-1) = max(0, -1) = 0
> > - f(1) = max(0, 1) = 1
> > - La suma de ambos resultados es 0 + 1 = 1
>
> Pero si primero sumamos los valores internamente:
>
> - f(-1 + 1) = f(0) = max(0, 0) = 0
>
> Como 1 no es igual a 0, el principio de superposición falla. Ese punto exacto donde la línea horizontal se convierte repentinamente en una diagonal (x = 0) destruye la linealidad matemática de la función en todo su dominio. Técnicamente se le conoce como una función lineal a trozos (piecewise linear).
>
> ¿Por qué este "quiebre" es un superpoder? Si las redes neuronales utilizaran funciones 100% lineales sin quiebres (por ejemplo, f(x) = 2x), no importaría cuántas capas ocultas o millones de neuronas le agregues al modelo: al multiplicar líneas rectas por líneas rectas, todas las capas colapsarían matemáticamente en una sola ecuación de una recta simple. La red jamás podría aprender patrones curvos o complejos, como el contorno de una cara humana en una fotografía.
>
> Al usar ReLU, cada neurona introduce un quiebre en el espacio matemático. Cuando sumas y superpones millones de estas líneas dobladas a través de diferentes capas, la red neuronal gana la capacidad de "doblarse" alrededor de los datos y aproximar cualquier curva suave y compleja imaginable. Es el equivalente geométrico a construir un círculo perfecto utilizando miles de palillos rectos muy pequeños.

Article 1: https://www.datasciencecentral.com/profiles/blogs/deep-learning-advantages-of-relu-over-sigmoid-function-in-deep#:~:text=Advantage%3A,exponential%20operations%20as%20in%20Sigmoids

Article 2: https://datascience.stackexchange.com/questions/26475/why-is-relu-used-as-an-activation-function

Fuentes adicionales que pueden utilizar:

- https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6
- http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf
- https://www.youtube.com/watch?v=WTQaox7NKNA